

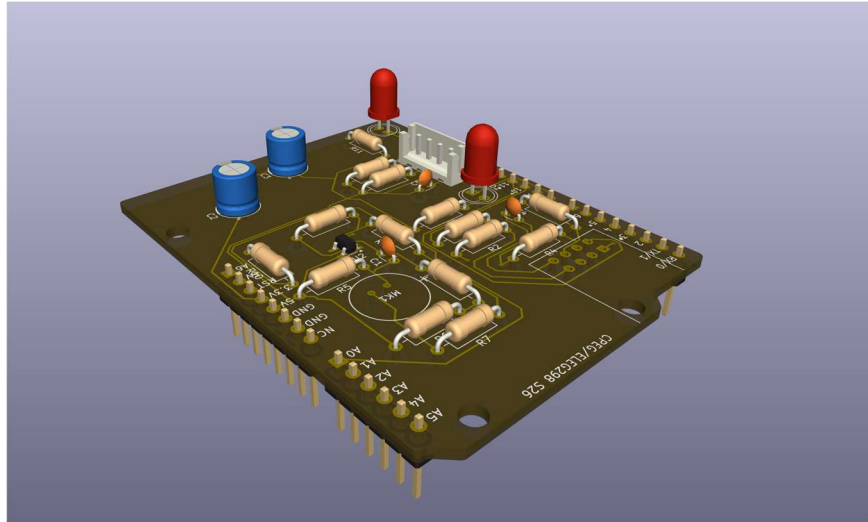
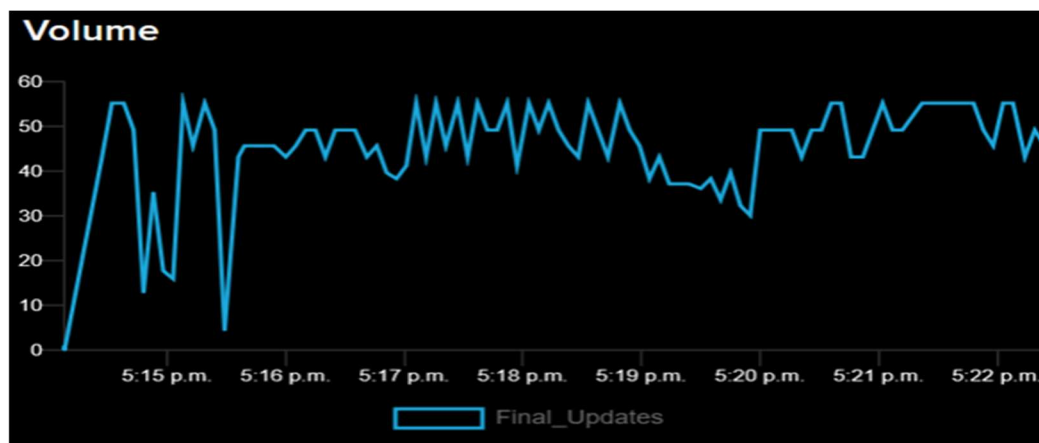


Image rendered by KiCAD

Venue Decibel Meter IoT Device

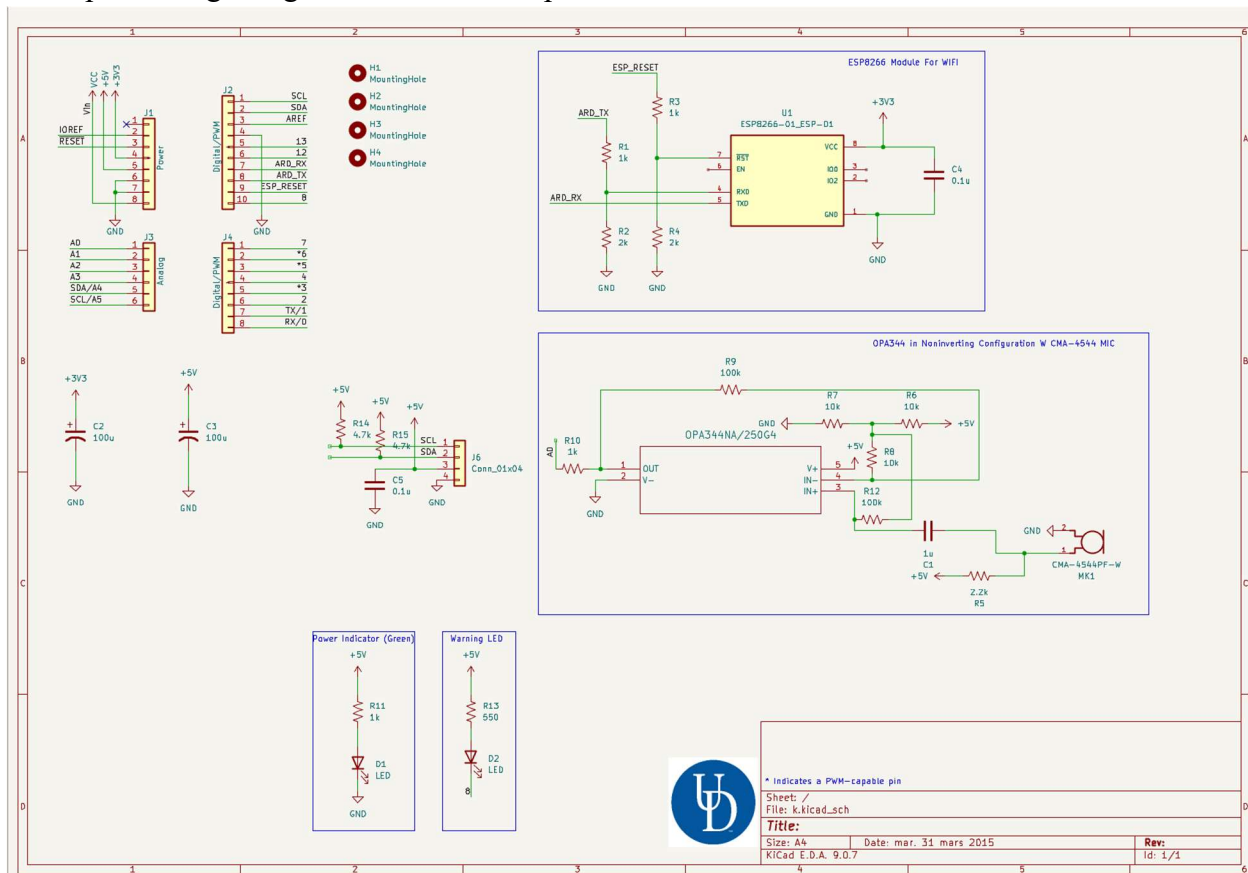


Project Description

The IoT shield takes the audio input via the microphone and amplifies the AC signal into an operational amplifier wired in non-inverting bias to create the maximum AC output. This is then taken into the Arduino UNO analog-to-digital converter to be processed as a digital signal to then take the average of peak-to-peak waves for the conversion to dB for accurate readings. This is then uploaded to the ESP-01S for updates through the Adafruit Dashboard. The direct AC signal is also connected into the indicator LED to show when the dB exceeds threshold value.

The Schematic

The main components used in the audio monitoring section of the project were the CMA-4544PF-W electret condenser microphone and the Texas Instruments OPA344 operational amplifier. The microphone was selected because of its large surplus of stock, low power requirements, and high sensitivity to environmental audio. The OPA344 operational amplifier was chosen because of its low input voltage operation and compact design, making it ideal for audio processing using the Arduino UNO platform shield.



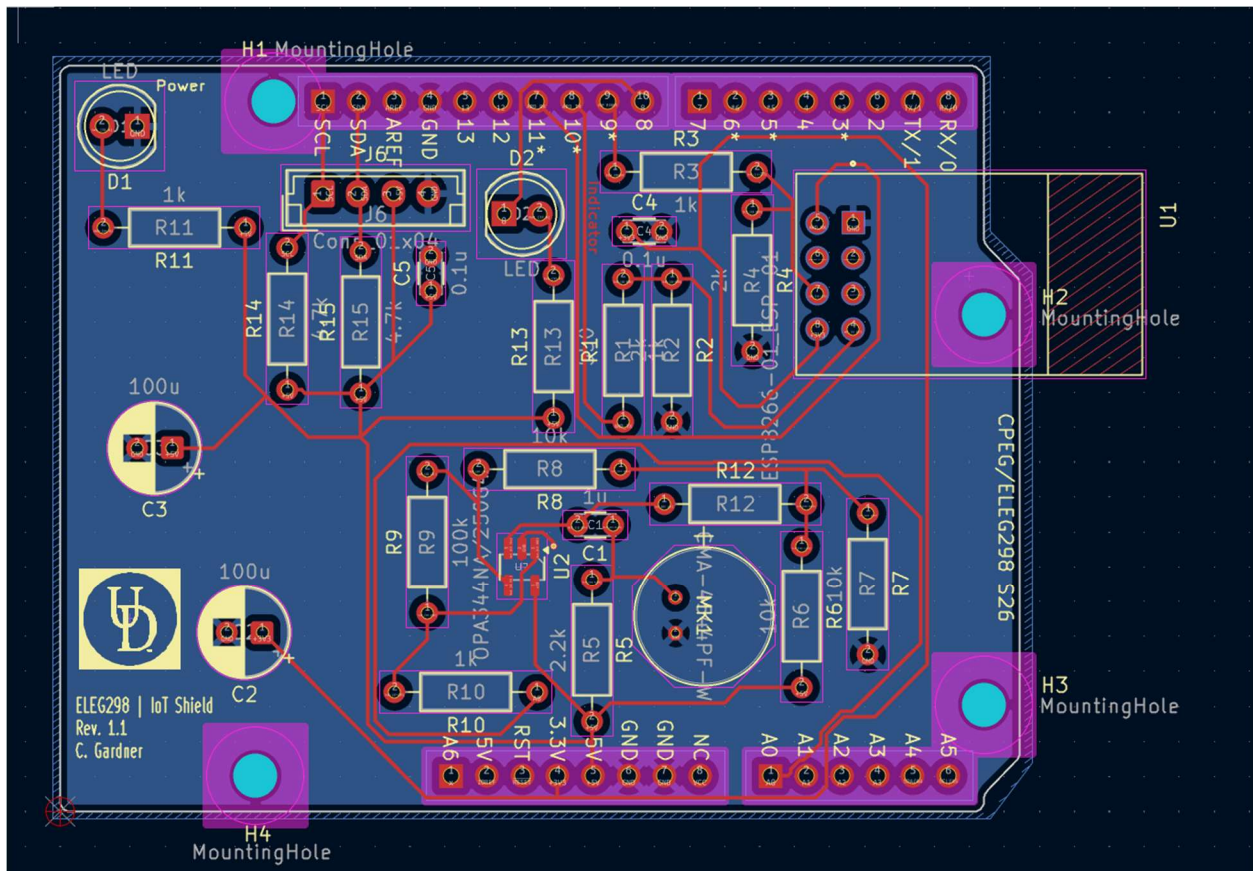
The microphone and amplifier system specifications are listed below:

- Operating Voltage (Microphone): 2 VDC typical
- Operating Voltage (OPA344): 2.5 V – 5.5 V
- Operating Current (OPA344): 250 μ A typical
- Gain Bandwidth Product: 1 MHz
- Rail-to-Rail Input and Output Operation
- Low Noise CMOS Architecture
- Electret Microphone Sensitivity: -44 dB typical
- Frequency Response Range: approximately 20 Hz – 20 kHz
- Warning Indicator Threshold: approximately 70 dB
- Operating Temperature Range for OPA344: approximately -40°C to +85°C

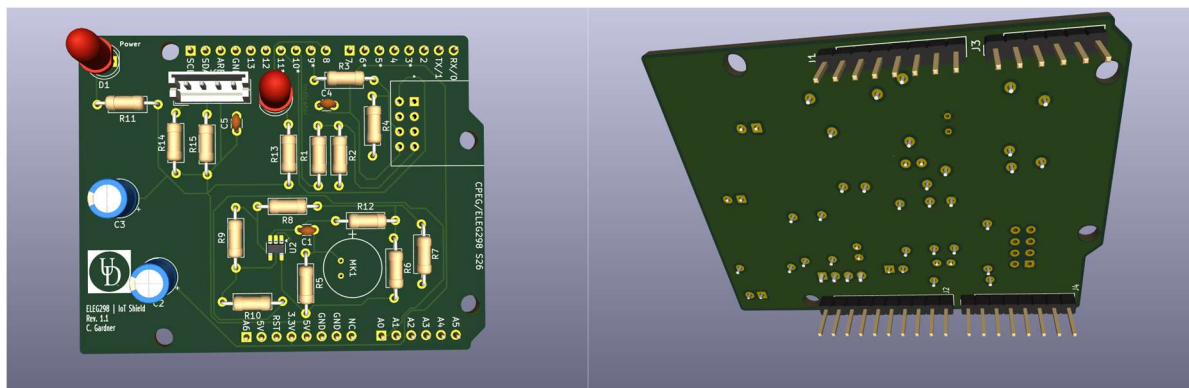
The operational amplifier uses resistor feedback to amplify the microphone signal while taking a biased signal for analog-to-digital conversion. The processed signal is continuously sampled by the Arduino UNO analog-to-digital converter and analyzed in software to estimate environmental sound pressure levels in decibels. The ESP8266 module then uploads the processed data to the Adafruit IO dashboard for monitoring and visualization.

The Printed Circuit Board Design

The printed circuit board designed in KiCad is shown below. The board is a two layer board with a ground plane for its bottom layer. It was based on the Arduino UNO platform to fit on top as a shield. The ESP-01S module is shown on the top right side, while the microphone and amplifier are seen in the bottom middle and all indicators positioned at the top.



The manufacturer images of the top and bottom of the PCB are shown in the KiCad 3D viewer. This allows us to see clearly the green solder mask color were picked with white accents. The PCB was manufactured by the JLCPCB Foundry.



Component List / Bill of Materials

All components are listed beside the PCB schematic of the Shield. Each Reference tells where each component is placed.

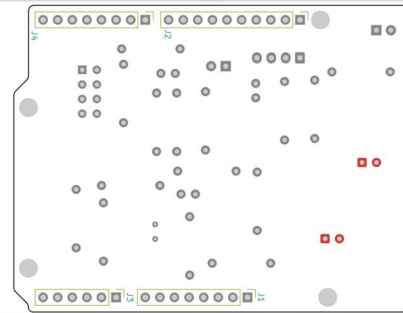
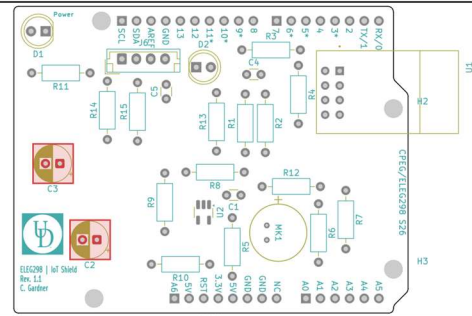
k

Rev:

mar. 31 mars 2015



Ref lookup		Filter				
	Sour ced	Plac ed	References	Value	Footprint	Quantity
1	☐	☐	C2, C3	100u	CP_Radial_D6.3mm_P2.50mm	2
2	☐	☐	C4, C5	0.1u	C_Disc_D3.0mm_W1.6mm_P2.50mm	2
3	☐	☐	C1	1u	C_Disc_D3.0mm_W1.6mm_P2.50mm	1
4	☐	☐	R1, R3, R10, R11	1k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	4
5	☐	☐	R6, R7, R8	10k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	3
6	☐	☐	R2, R4	2k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	2
7	☐	☐	R9, R12	100k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	2
8	☐	☐	R14, R15	4.7k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	2
9	☐	☐	R5	2.2k	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	1
10	☐	☐	R13	550	R_Axial_DIN0207_L6.3mm_D2.5mm_P1.0.16mm_Horizontal	1
11	☐	☐	D1, D2	LED	LED_D5.0mm	2
12	☐	☐	U1	ESP8266-01_ESP-01	XCVR_ESP8266-01_ESP-01	1
13	☐	☐	U2	OPA344NA/250G4	SOT-23-5_HandSoldering	1
14	☐	☐	MK1	CMA-4544PF-W	CMA_CUI_9P7X9P7_CUD	1
15	☐	☐	J1	Power	PinHeader_1x08_P2.54mm_Vertical	1
16	☐	☐	J2	Digital/PWM	PinHeader_1x10_P2.54mm_Vertical	1
17	☐	☐	J3	Analog	PinHeader_1x06_P2.54mm_Vertical	1
18	☐	☐	J4	Digital/PWM	PinHeader_1x08_P2.54mm_Vertical	1
19	☐	☐	J6	Conn_01x04	JST_EH_848-EH-A_1x04_P2.50mm_Vertical	1



Most parts were sourced from the University of Delaware Undergraduate Laboratory however; two pieces were acquired from DigiKey. Details listed below.

Component	Part #	Price Per	Link
ESP8266-01	3647-26209-ND	\$8.84	https://www.digikey.com/en/products/detail/universal-solder-electronics-ltd/26209/14319860
OPA344PA	OPA344NA/250	\$2.12	https://www.digikey.com/en/products/detail/texas-instruments/OPA344NA-250/362264
CMA-4544PF	102-1721-ND	\$0.93	https://www.digikey.com/en/products/detail/samsky-formerly-cui-devices/CMA-4544PF-W/1869981

Arduino Code and Libraries

The full code is listed in the Appendix however, here is the libraries used:

```
#include "Arduino.h"
```

```
#include <SoftwareSerial.h>
```

The defined pins and addresses are as follows:

```
#define reset_pin 9
```

```
#define tx_pin 10
```

```
#define rx_pin 11
```

The setup function establishes serial communication between the Arduino UNO and ESP8266 Wi-Fi module while initializing the microphone analog input and GPIO pins. The ESP8266 then connects to the network and establishes communication with Adafruit IO for monitoring. Once the connection succeeds, the required publish feeds are initialized before the program enters the main loop.

```
response = espData("setup_pubfeed=1,CPEG298pub", 8000, true);
```

```
response = espData("setup_subfeed=3,reset_button", 8000, true);
```

```
response = espData("setup_time,-5", 8000, true);
```

```
Serial.println("Setup Complete");
```

```
response = espData("get_name=1", 1000, true);
```

```
response = espData("get_name=3", 1000, true);
```

```
response = espData("get_date", 1000, true);
```

```
response = espData("get_time", 1000, true);
```

```
response = espData("send_data=3,0", 4000, true);
```

Here is the corresponding serial monitor that is fed directly from the edge device:

Resp: S26.1.0 (2/08/26)

Resp: WiFi SSID:Rob

Resp: IO User:garden25

Resp: Setting up IO.....Adafruit IO connected.

Resp: PubFeed 1 setup

Resp:

Resp:

Setup Complete

Resp:

Resp:

Resp:

Resp:

Resp:

AC ADC value is: 3

AC voltage is: 0.0147

dB value is: 45.71

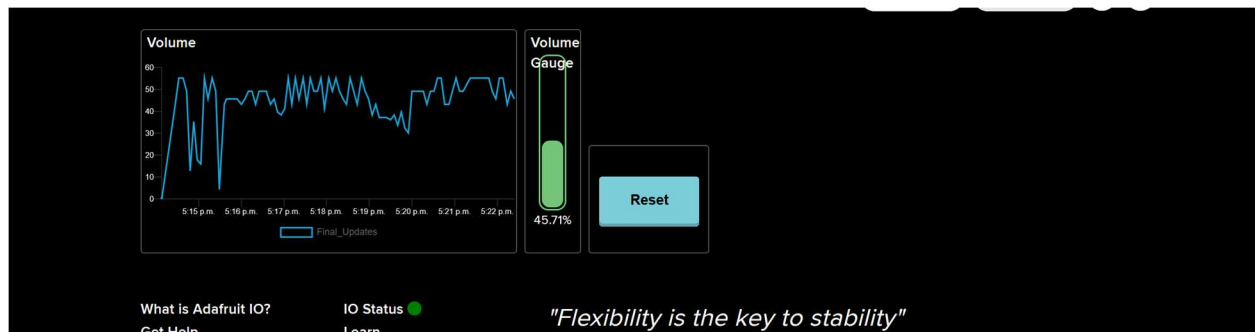
AC ADC value is: 2

AC voltage is: 0.0098

dB value is: 49.23

Adafruit IO dashboard

Below is the Adafruit IO Dashboard with its data showing volume compared to maximum to local statue, a graph overtime of volume, and a reset button for every use.



Conclusion

Overall, the Venue Audio Monitor IoT Device successfully demonstrated real time audio monitoring and wireless communication through Adafruit IO. One major issue encountered during PCB design was accidentally wiring the warning LED directly to 5 V, which prevented it from properly functioning as a sound threshold indicator until the circuit was manually modified during prototyping by connecting the positive terminals to pin A0 and GND`1 to the ground pin. In the future, I would like to redesign the PCB to correct the LED circuitry, improve the dB conversion accuracy, and add additional sensors or data logging capabilities for long-term venue monitoring.

Appendix

```
#include "Arduino.h"
```

```
#include <SoftwareSerial.h>
```

```
#define reset_pin 9
```

```
#define tx_pin 10
```

```

#define rx_pin 11

SoftwareSerial espSerial(rx_pin, tx_pin);

String response;
String resp;

const int sensorpin = A0;

String espData(String command, const long timeout, boolean debug);

void setup() {

    Serial.begin(115200);
    espSerial.begin(9600);

    pinMode(sensorpin, INPUT);

    pinMode(reset_pin, OUTPUT);
    digitalWrite(reset_pin, LOW);
    delay(500);
    digitalWrite(reset_pin, HIGH);

    Serial.println("\n\nCPEG/ELEG Demo S26 rev1.0");
    Serial.println("setting up Adafruit IO");

    response = espData("get_macaddr", 2000, true);
    response = espData("get_version", 2000, true);
    response = espData("wifi_ssid=" + WIFI_SSID, 4000, true);
    response = espData("wifi_pass=" + WIFI_PASS, 4000, false);

```



```

response = espData("io_key=" + IO_KEY, 4000, false);
response = espData("io_user=" + IO_USERNAME, 4000, true);

response = espData("setup_io", 30000, true);

if (response.indexOf("connected") < 0) {
    Serial.println("\nAdafruit IO Connection Failed");
    while (1);
}

response = espData("setup_pubfeed=1,CPEG298pub", 8000, true);
response = espData("setup_subfeed=3,reset_button", 8000, true);
response = espData("setup_time,-5", 8000, true);

Serial.println("Setup Complete");

response = espData("get_name=1", 1000, true);
response = espData("get_name=3", 1000, true);
response = espData("get_date", 1000, true);
response = espData("get_time", 1000, true);

response = espData("send_data=3,0", 4000, true);
}

```

```

void loop() {

```

```

    int minValue = 1023;
    int maxValue = 0;

```

```

unsigned long startTime = millis();

while (millis() - startTime < 100) {
    int reading = analogRead(sensorpin);

    if (reading < minValue) minValue = reading;
    if (reading > maxValue) maxValue = reading;
}

int peakToPeak = maxValue - minValue;

float voltagePP = peakToPeak * (5.0 / 1023.0);
float voltageRMS = voltagePP / (2.0 * 1.414);

float dB = ( 20.0 * log10(voltageRMS / 1.0)) * -1;
// chatgpt used to generate math 'generate a arduino code to take input AC voltage and convert
to dB'

Serial.print("AC ADC value is: ");
Serial.println(peakToPeak);

Serial.print("AC voltage is: ");
Serial.println(voltagePP, 4);

Serial.print("dB value is: ");
Serial.println(dB, 2);

response = espData("send_data=1," + String(dB, 2), 4000, false);

delay(5000);

```

```
}
```

```
String espData(String command, const long timeout, boolean debug) {
```

```
    char c = 0;
```

```
    response = "";
```

```
    espSerial.println(command);
```

```
    long int time = millis();
```

```
    while ((time + timeout) > millis()) {
```

```
        while (espSerial.available()) {
```

```
            c = espSerial.read();
```

```
            response += c;
```

```
        }
```

```
        if ((c == '\n') && (command != "setup_io")) break;
```

```
    }
```

```
    response.trim();
```

```
    if (debug) {
```

```
        Serial.println("Resp: " + response);
```

```
    }
```

```
    return response;
```

```
}
```